

Project 1: Minesweeper Documentation

Development Environment

- Programming Language: Python 3.13.15
- Game Library: Pygame
- Pygame must be installed to run minesweeper.py

Person-Hours Estimate:

Team Member	Assigned Task	Estimated Hours
Parker	Coding	5
Aidan	Coding	5
Joseph	Coding	5
Ever	Documentation	2
Atique	Documentation	2
Viren	Documentation	2

Estimation Methodology

During the 2nd team meeting after we divided the coding and documentation responsibilities, each team member gave an estimate on how much time they expect their assigned tasks to take.

Actual-Person Hours:

Date	Team Member	Work	Start	Stop	Time Spent
9/16	Aidan	general playing field generation	1:28 pm	3:39 pm	2 hours 11 min
9/16	Aidan	continued general field generation	6:19 pm	9:00 pm	2 hours 41 min
9/16	Ever	Project documentation doc created	5:31 pm	6:15 pm	44 min
9/16	Ever	Continued documentation	7:15 pm	7:47 pm	32 min
9/17	Parker	Working on code	10:10 am	11:16 am	1 hour 6 min
9/17	Atique	Filled some System Arch	9:44 am	10:44 am	1 hour
9/17	Parker	bombs	12:00 pm (?)	2:10 pm	2 hours 10 min
9/17	Parker	Bug fix	2:10 pm	2:31 pm	21 minutes

9/18	Viren	Github commits and repo updates	9.35 pm	10.15 pm	40 Minutes
------	-------	---------------------------------	---------	----------	------------

9/19	Viren	Continued updating github repo, merge commits, adding to google doc.	6.30 pm	7.50 pm	1 hour 20 min
------	-------	--	---------	---------	---------------

System Architecture Overview:

High Level Description:

The Minesweeper application is implemented in Python using the Pygame library. The program creates a 10 × 10 game board made up of Tile objects. Pygame handles the game window, graphics, mouse input, and screen updates. Each tile stores its position and current state. The main game loop detects user input, updates the appropriate tile, and redraws the board to display the updated game state.

Components:

Tile Class: Represents an individual tile on the Minesweeper board. Each Tile stores its x and y coordinates, its rectangular screen area, and its current state.

Game Board: A two-dimensional list containing the Tile objects that make up the 10 × 10 playing field.

getTile(): Determines which tile is located under the user's mouse cursor.

drawBoard(): Draws the game board, row and column labels, and the correct image for each tile based on its current state.

Main Game Loop: Processes Pygame events such as closing the window, mouse movement, left clicks, and right clicks. It then redraws the game board.

Data Flow:

The program begins by initializing Pygame, loading the tile images, and creating the game board. The program then enters the main game loop and waits for user input. When the mouse moves or a mouse button is clicked, getTile() determines which tile the user is interacting with. A left click changes a covered tile to a revealed state, while a right click adds or removes a flag. After processing the input, drawBoard() uses the current tile states to redraw the game board, and Pygame updates the display.

The current event handling supports left-click revealing and right-click flag toggling.

A very simple flow is:

User Input → Pygame Event → getTile() → Update Tile State → drawBoard() → Updated Game Window

Key Data Structures:

- The Minesweeper program we used has many important key data structures to manage and store the game properties.
The main data structure in the program is the “**Tile Class**”. Each tile stores its x and y position on the board, its rectangular screen area, current state, and whether it contains a bomb or not.
- The board itself is stored as a 2-dimensional list of **Tile** objects. The board contains 10 rows and 10 columns, creating a 10 x 10 play board/ field.
- Every tile uses a numerical state value to determine how the tile should be displayed. A value of **-1** shows a covered tile, **-2** means a flagged tile, **-3** for a revealed bomb, **0** represents a blank revealed tile and values from **1 to 8** for representing number of bombs touching/ surrounding that specific tile.
- The program uses the **numbers dictionary** to connect the values 1 through 8 with the correct image, which will be displayed for each numbered tile.
- When **bombs** are created, the program builds a temporary list containing every tile except the player’s first selected tile. **Random Tiles** are then selected from the list.
- Lastly, the “**reveal_tile()**” function uses a **temporary list** as a stack when revealing empty areas of the board. If a blank tile is found, then the nearby tiles are later added to the stack and checked until all connected blank areas are all revealed.